ENV 797 Forecasting Competition Report

Github Link:
https://github.com/sylviahipp/HippYe_ENV797_TSA_ForecastCompetition_S26

Sylvia Hipp and Janice Ye

April 24, 2026

Setup Environment

Load Packages

Import and Wrangle Data

Load Data

```
load_df_raw <- read_excel(here("data/raw/load.xlsx"))

# Processing steps:
# pivot longer to hourly data
# group by date and take average load over 24-hours

load_df <- load_df_raw %>%
  mutate(date = as.Date(date)) %>%
  pivot_longer(cols = c(h1:h24),
               names_to = "hour", values_to = "load") %>%
  group_by(date) %>%
  # daily load = average across all 24 hours
  summarize(daily_load = mean(load, na.rm = TRUE)) %>%    # exclude NAs when calculating load
  ungroup()
```

Weather Data

```
# Import raw data
humidity_raw <- read_excel(here("data/raw/relative_humidity.xlsx"))
temperature_raw <- read_excel(here("data/raw/temperature.xlsx"))

# Clean data ---------------------------------------------------------------

# get average temp and humidity values for each hour

rh_df <- humidity_raw %>%
```

```r
  mutate(date = as.Date(date),
         datetime = as.POSIXct(glue("{date} {hr}:00"))) %>%
  pivot_longer(cols = contains("rh"), values_to = "relative_humidity") %>%
  group_by(date) %>%
  summarize(avg_rh = mean(relative_humidity)) %>%
  ungroup()

#summary(humidity_avg)  # no missing values

temp_df <- temperature_raw %>%
  filter(!is.na(date)) %>%
  mutate(date = as.Date(date),
         datetime = as.POSIXct(glue("{date} {hr}:00"))) %>%
  pivot_longer(cols = contains("t_"), values_to = "temperature") %>%
  group_by(date) %>%
  summarize(avg_temp = mean(temperature)) %>%
  ungroup()

#summary(temperature_avg)  # no missing values
```

Combine Datasets and convert to time series

Combine

```r
# Combine datasets
comb <- load_df %>%
  left_join(temp_df, by = "date") %>%
  left_join(rh_df, by = "date")

# Make sure there are no missing dates
start_date <- first(comb$date)
end_date <- last(comb$date)

dates_seq <- seq.Date(start_date, end_date, by = "day") %>% tibble() %>% rename("date" = '.')

comb <- dates_seq %>%
  left_join(comb)

#summary(comb)  # no NAs
```

Split into test and training period

```r
start_test <- as.Date("2011-06-01")
start_test_doy <- as.numeric(format(start_test, "%j"))
end_test <- as.Date("2011-06-30")


train <- comb %>%
  filter(date >= start_date,
```

```
         date < start_test)

test <- comb %>%
  filter(date >= start_test,
         date <= end_test)

full <- comb %>%
  filter(date >= start_date,
         date <= end_date)
```

Convert to time series

```
train_ts <- msts(train$daily_load,
                 seasonal.periods = c(7, 365.25),
                 start = c(year(start_date), 1))

test_ts <- msts(test$daily_load,
                seasonal.periods = c(7, 365.25),
                start = c(year(start_test), start_test_doy))

full_ts <- msts(full$daily_load,
                seasonal.periods = c(7, 365.25),
                start = c(year(start_date), 1))
```

Load

```
temp_ts_train <- msts(train$avg_temp,
                      seasonal.periods = c(7, 365.25),
                      start = c(year(start_date), 1))

temp_ts_full <- msts(full$avg_temp,
                     seasonal.periods = c(7, 365.25),
                     start = c(year(start_date), 1))

rh_ts_train <- msts(train$avg_rh,
                      seasonal.periods = c(7, 365.25),
                      start = c(year(start_date), 1))

rh_ts_full <- msts(full$avg_rh,
                     seasonal.periods = c(7, 365.25),
                     start = c(year(start_date), 1))
```

Weather

Fit Time Series Models

Setup

```
# designate length of forecast
n_forecast = length(test_ts)
```

Get Seasonal Naive for Weather Data for forecasting

```
temp_snaive_test <- snaive(temp_ts_train, h = n_forecast)
rh_snaive_test <- snaive(rh_ts_train, h = n_forecast)
```

Model 1: ARIMA + Fourier (Score = 0.0960)

```
# find the most applicable K
best_dhr <- list(
  rmse = Inf,
  K_week = NA,
  K_year = NA,
  fit = NULL,
  fc = NULL
)

# testing
for (K_week in 1:2) {
  for (K_year in 3:6) {
    xreg_train <- fourier(train_ts, K = c(K_week, K_year))
    xreg_test <- fourier(train_ts, K = c(K_week, K_year), h = 31)

    fit_try <- auto.arima(
      train_ts,
      xreg = xreg_train,
      seasonal = FALSE,
      stepwise = TRUE,
      approximation = TRUE,
      lambda = "auto",
      biasadj = TRUE
    )

    fc_try <- forecast(fit_try, xreg = xreg_test, h = 31)

    rmse_try <- accuracy(as.numeric(fc_try$mean), as.numeric(test$daily_load))["Test set", "RMSE"]

    if (rmse_try < best_dhr$rmse) {
      best_dhr$rmse <- rmse_try
      best_dhr$K_week <- K_week
      best_dhr$K_year <- K_year
      best_dhr$fit <- fit_try
      best_dhr$fc <- fc_try
    }
```

```r
  }
}

# check best dhr
best_dhr$K_week
```

```
## [1] 1
```

```r
best_dhr$K_year
```

```
## [1] 4
```

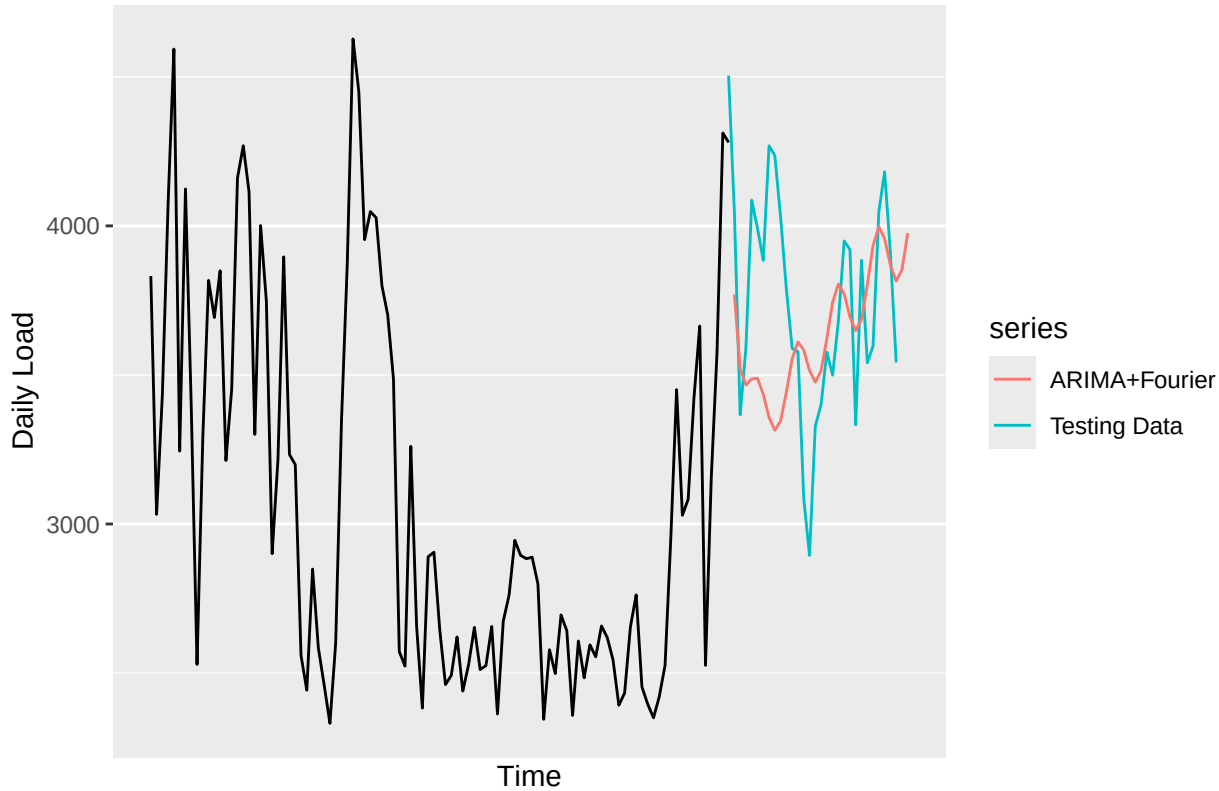
```r
best_dhr$rmse
```

```
## [1] 424.8978
```

```r
accuracy(as.numeric(best_dhr$fc$mean), as.numeric(test_ts))
```

```
##                 ME     RMSE      MAE      MPE     MAPE
## Test set 108.4046 424.8978 346.1554 1.960412 9.113534
```

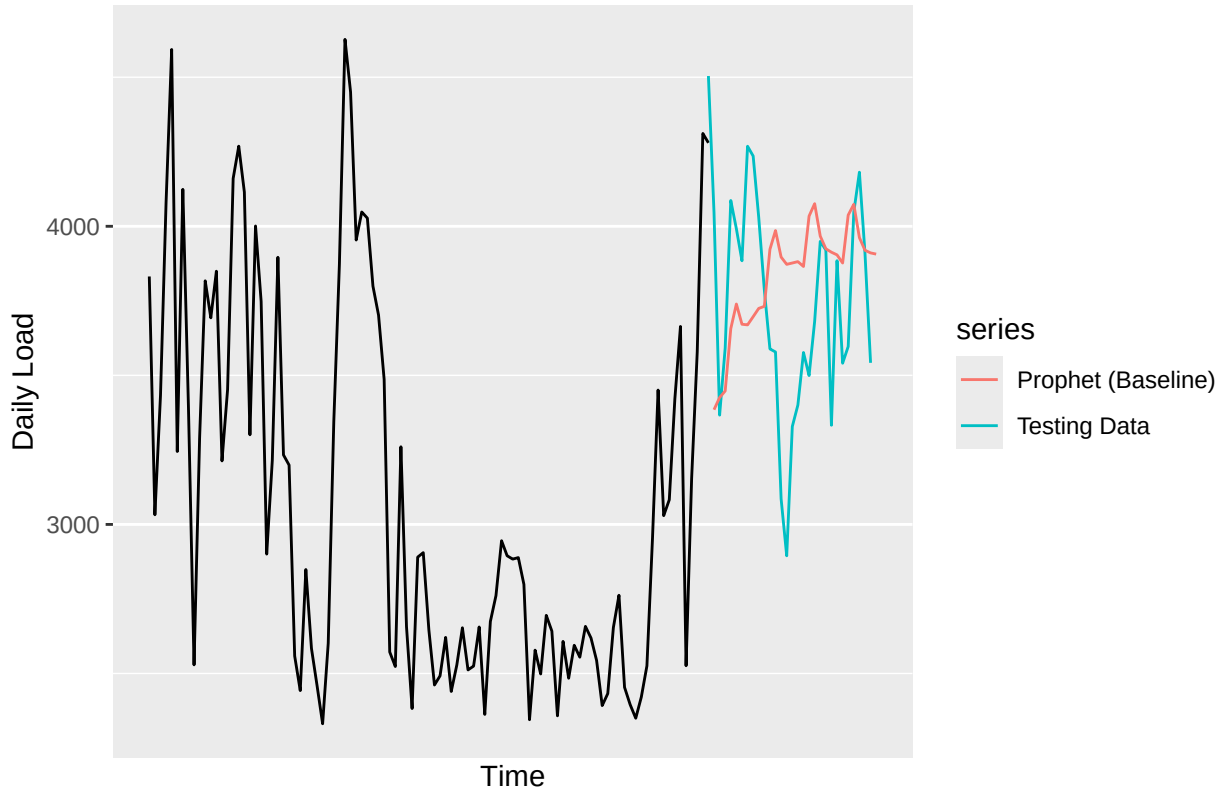
```r
K <- c(best_dhr$K_week, best_dhr$K_year)

# plot
autoplot(train_ts %>% subset(start = length(train_ts)-100)) +
  autolayer(test_ts, series = "Testing Data") +
  autolayer(best_dhr$fc$mean, series = "ARIMA+Fourier") +
  ylab("Daily Load")
```



Model 2: Prophet Baseline (Score = 0.0973)

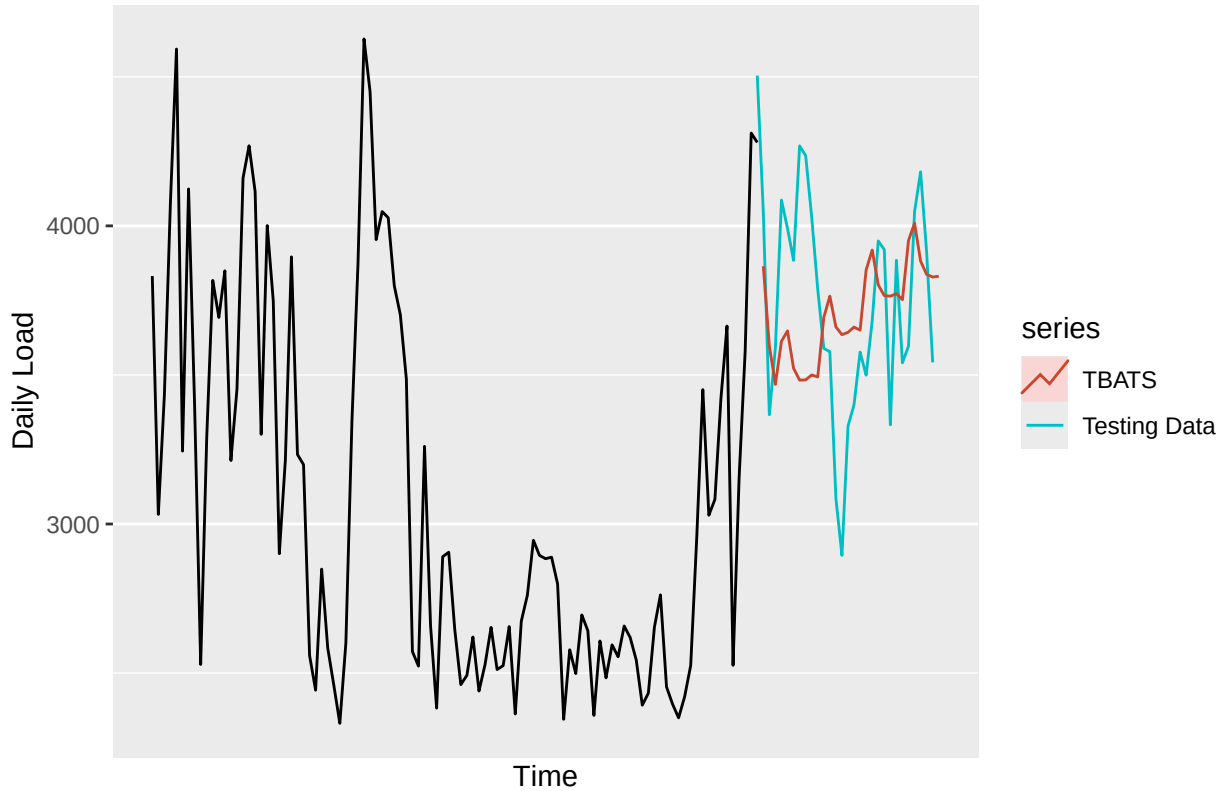
```
# Prepare data
proph_load_train <- train %>%
  select(date, daily_load) %>%
  rename(ds=date,y=daily_load)

# Fit model
proph_fit_load <- prophet(proph_load_train)

# Create future data frame
proph_future <- make_future_dataframe(proph_fit_load, periods = n_forecast)

# Fit model
proph_load_forecast <- predict(proph_fit_load,proph_future)

# convert forecast in ts object
prophet_for_ts <- msts(proph_load_forecast$yhat,
                       start=c(year(start_date), 1),
                       seasonal.periods=c(7,365.25))

prophet_for_ts_test <- tail(prophet_for_ts,n_forecast)

# Plot model + observed data
```

```r
autoplot(train_ts %>% subset(start = length(train_ts)-100)) +
  autolayer(test_ts, series = "Testing Data") +
  autolayer(prophet_for_ts_test, series="Prophet (Baseline)",PI=FALSE) +
  ylab("Daily Load")
```



Model 3: TBATS (Score = 0.0998)

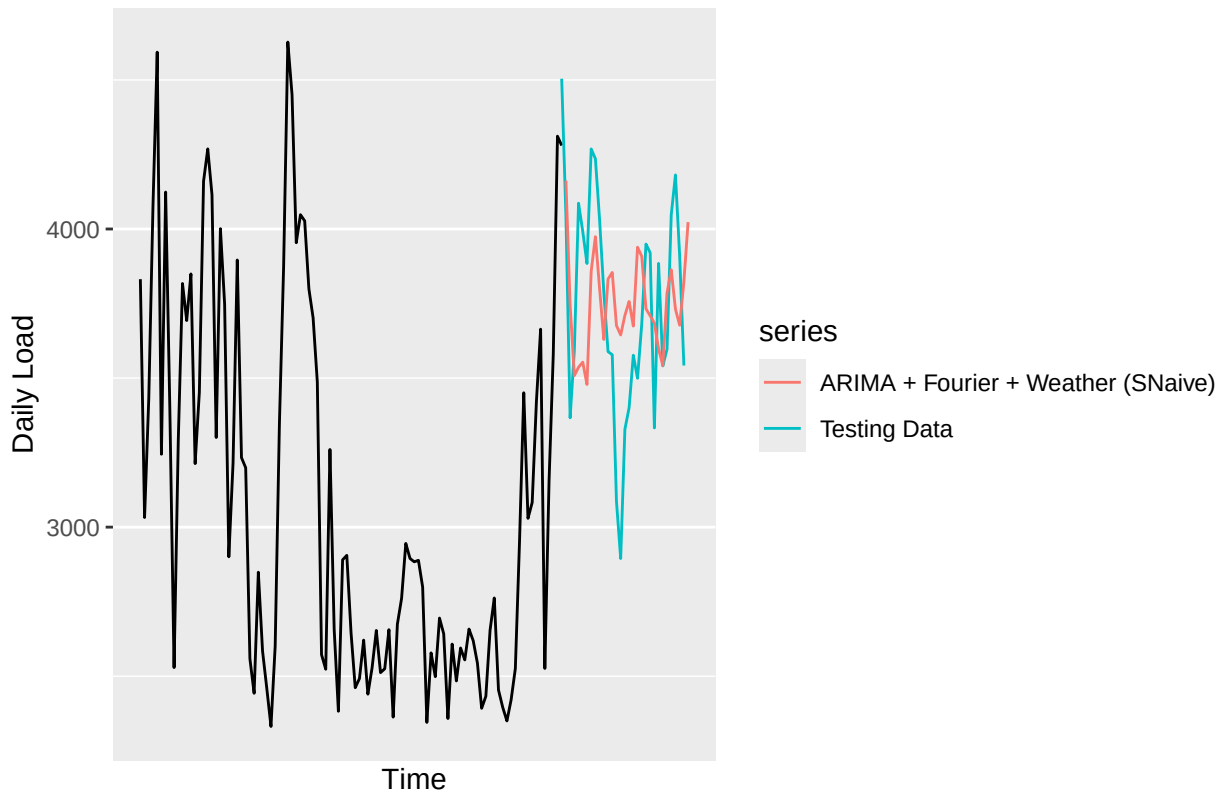
```r
# Fit TBATA model
tbats_fit_train <- tbats(train_ts)

tbats_forecast_train <- forecast(tbats_fit_train, h=n_forecast)

# Plot model + observed data
autoplot(train_ts %>% subset(start = length(train_ts)-100)) +
  autolayer(test_ts, series = "Testing Data") +
  autolayer(tbats_forecast_train, series="TBATS",PI=FALSE) +
  ylab("Daily Load")
```



Model 4: ARIMA + Fourier + Weather (Snaive Forecast) (score = 0.1018)

```
xreg_rh_temp_train <- cbind(fourier(train_ts, K=c(2,12)),
                            rh = rh_ts_train,
                            temp = temp_ts_train)
xreg_rh_temp_forecast <- cbind(fourier(train_ts, K=c(2,12), h=n_forecast),
                               rh = rh_snaive_test$mean,
                               temp = temp_snaive_test$mean)

# FORCE matching column names and order
colnames(xreg_rh_temp_forecast) <- colnames(xreg_rh_temp_train)
xreg_rh_temp_forecast <- xreg_rh_temp_forecast[, colnames(xreg_rh_temp_train)]

# Convert to matrix
xreg_rh_temp_train <- as.matrix(xreg_rh_temp_train)
xreg_rh_temp_forecast <- as.matrix(xreg_rh_temp_forecast)

arima_rh_temp_train <- auto.arima(train_ts,
                                  seasonal = FALSE,
                                  lambda = NULL,
                                  xreg = xreg_rh_temp_train)

arima_rh_temp_forecast <- forecast(arima_rh_temp_train,
                                   xreg = xreg_rh_temp_forecast,
```

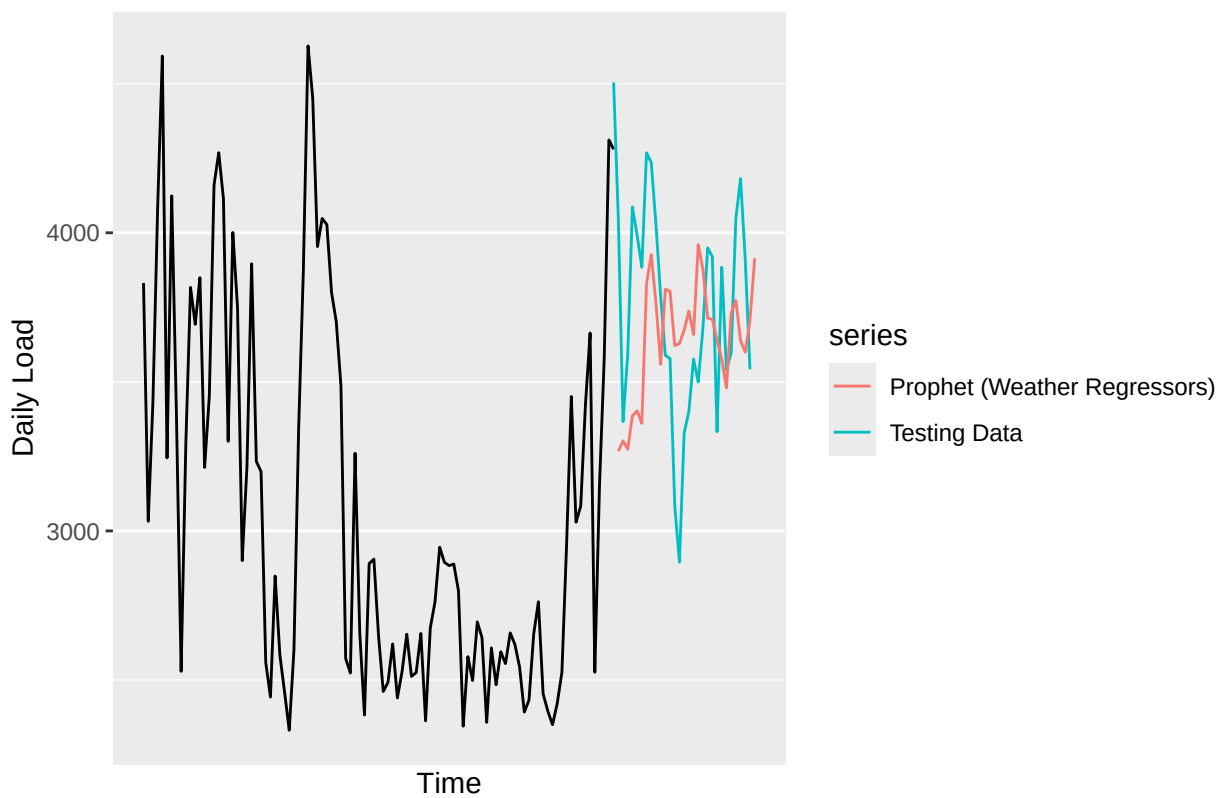
```
                                    h = n_forecast)

# summary(arima_four_model_train)
#checkresiduals(arima_rh_temp_train)   # appears near white noise series

# Plot model + observed data
autoplot(train_ts %>% subset(start = length(train_ts)-100)) +
  autolayer(test_ts, series = "Testing Data") +
  autolayer(arima_rh_temp_forecast$mean, series="ARIMA + Fourier + Weather (SNaive)") +
  ylab("Daily Load")
```



Model 5: Prophet + Weather (Snaive Forecast) (Score = 0.1056)

```
prophet_load_weather_train <- train %>%
  rename(ds = date, y = daily_load, temp = avg_temp, rh = avg_rh)

# create model -- used ChatGPT to learn how to add extra regressors
prophet_m_regressors <- prophet() %>%
  add_regressor("temp", standardize = TRUE) %>%
  add_regressor("rh", standardize = TRUE)

# fit model to training data
prophet_fit_regressors <- fit.prophet(prophet_m_regressors,
```

```r
                                        prophet_load_weather_train)

# Create future data frame
proph_future_regressors <- make_future_dataframe(prophet_fit_regressors,
                                                 periods = n_forecast)

# add regressors to future data frame
proph_future_regressors$temp <- c(temp_ts_train, temp_snaive_test$mean)
proph_future_regressors$rh <- c(rh_ts_train, rh_snaive_test$mean)

# Predict
proph_load_weather_forecast <- predict(prophet_fit_regressors,proph_future_regressors)

# convert forecast in ts object
prophet_for_weather_ts <- msts(proph_load_weather_forecast$yhat,
                               start=c(year(start_date), 1),
                               seasonal.periods=c(7,365.25))

prophet_for_weather_ts_test <- tail(prophet_for_weather_ts,n_forecast)

# Plot model + observed data
autoplot(train_ts %>% subset(start = length(train_ts)-100)) +
  autolayer(test_ts, series = "Testing Data") +
  autolayer(prophet_for_weather_ts_test, series="Prophet (Weather Regressors)") +
  ylab("Daily Load")
```



Compare Model Performance

We trained our model on data over the period January 1, 2005–May 31, 2011. The model was then tested on its performance to predict load in June 2011. For our submissions for the competition, we trained the models on the entire dataset then projected load for the month of July 2011.

```
# Get all accuracy scores
arima_fourier_scores <- accuracy(best_dhr$fc$mean, test_ts)
tbats_scores <- accuracy(tbats_forecast_train$mean, test_ts)
prophet_scores <- accuracy(prophet_for_ts_test, test_ts)
arima_regressor_scores <- accuracy(arima_rh_temp_forecast$mean, test_ts)
prophet_weather_scores <- accuracy(prophet_for_weather_ts_test, test_ts)
```

```
# Combine scores in table
names <- colnames(tbats_scores)

scores <- rbind(
  arima_fourier_scores,
  prophet_scores,
  prophet_weather_scores,
  arima_regressor_scores,
  tbats_scores
) %>% as.data.frame()

rownames(scores) <- c("Arima + Fourier", "Prophet (baseline)", "Prophet (+Weather Regressors)", "Arima +

# Display table with scores
scores %>%
  kable() %>%
  #row_spec(0, align = "c", bold = TRUE) %>%
  #column_spec(1, bold = TRUE) %>%
  kable_classic() %>%
  kable_styling(latex_options = "scale_down")
```

	ME	RMSE	MAE	MPE	MAPE	ACF1	Theil's U
Arima + Fourier	89.69203	392.7772	307.9481	1.585506	8.227726	0.6644091	1.179250
Prophet (baseline)	-111.07368	426.6303	346.8213	-3.965194	9.738730	0.6368094	1.354010
Prophet (+Weather Regressors)	83.24857	391.8613	341.4207	1.393345	9.244408	0.5853083	1.169543
Arima + Regressors + TBATS	-19.27064	343.0613	303.3830	-1.323406	8.386410	0.5839792	1.111579
TBATS	10.74461	371.1882	311.3241	-0.574993	8.508260	0.6484341	1.158888